```
from random import randint

N = 4

def configureRandomly(board, state):
    for i in range(N):
        state[i] = randint(0, 100000) % N
        board[state[i]][i] = 1

def printBoard(board):
    for i in range(N):
        print(*board[i])

def printState(state):
    print(*state)

def compareStates(state1, state2):
    for i in range(N):
        if state1[i] != state2[i]:
            return False
    return True

def fill(board, value):
    for i in range(N):
        for j in range(N):
            board[i][j] = value

def calculateObjective(board, state):
    attacking = 0
    for i in range(N):
        row = state[i]
        col = i - 1
        while col >= 0 and board[row][col] != 1:
            col -= 1
        if col >= 0 and board[row][col] == 1:
            attacking += 1

        row = state[i]
        col = i + 1
        while col < N and board[row][col] != 1:
            col += 1
        if col < N and board[row][col] == 1:
            attacking += 1

        row = state[i] - 1
        col = i - 1
        while col >= 0 and row >= 0 and board[row][col] != 1:
            col -= 1
            row -= 1
        if col >= 0 and row >= 0 and board[row][col] == 1:
            attacking += 1

        row = state[i] + 1
        col = i + 1
        while col < N and row < N and board[row][col] != 1:
            col += 1
            row += 1
        if col < N and row < N and board[row][col] == 1:
            attacking += 1

        row = state[i] + 1
        col = i - 1
        while col >= 0 and row < N and board[row][col] != 1:
            col -= 1
            row += 1
        if col >= 0 and row < N and board[row][col] == 1:
            attacking += 1

        row = state[i] - 1
        col = i + 1
        while col < N and row >= 0 and board[row][col] != 1:
            col += 1
            row -= 1
        if col < N and row >= 0 and board[row][col] == 1:
            attacking += 1

    return int(attacking / 2)

def generateBoard(board, state):
    fill(board, 0)
    for i in range(N):
```

```
        board[state[i]][i] = 1

def copyState(state1, state2):
    for i in range(N):
        state1[i] = state2[i]

def getNeighbour(board, state):
    opBoard = [[0 for _ in range(N)] for _ in range(N)]
    opState = [0 for _ in range(N)]

    copyState(opState, state)
    generateBoard(opBoard, opState)

    opObjective = calculateObjective(opBoard, opState)

    NeighbourBoard = [[0 for _ in range(N)] for _ in range(N)]
    NeighbourState = [0 for _ in range(N)]
    copyState(NeighbourState, state)
    generateBoard(NeighbourBoard, NeighbourState)

    for i in range(N):
        for j in range(N):
            if j != state[i]:
                NeighbourState[i] = j
                NeighbourBoard[NeighbourState[i]][i] = 1
                NeighbourBoard[state[i]][i] = 0

                temp = calculateObjective(NeighbourBoard, NeighbourState)

                if temp <= opObjective:
                    opObjective = temp
                    copyState(opState, NeighbourState)
                    generateBoard(opBoard, opState)

                NeighbourBoard[NeighbourState[i]][i] = 0
                NeighbourState[i] = state[i]
                NeighbourBoard[state[i]][i] = 1

    copyState(state, opState)
    fill(board, 0)
    generateBoard(board, state)

def hillClimbing(board, state):
    neighbourBoard = [[0 for _ in range(N)] for _ in range(N)]
    neighbourState = [0 for _ in range(N)]

    copyState(neighbourState, state)
    generateBoard(neighbourBoard, neighbourState)

    step = 1
    while True:
        copyState(state, neighbourState)
        generateBoard(board, state)

        current_objective = calculateObjective(board, state)
        print(f"Step {step}:")
        print(f"  Current State Permutation (columns 0 to {N-1}): {state}")
        print(f"  Attacking queen pairs: {current_objective}")
        print("  Board layout:")
        printBoard(board)
        print("-" * 40)

        getNeighbour(neighbourBoard, neighbourState)

        if compareStates(state, neighbourState):
            print("Final solution reached!")
            break
        elif calculateObjective(board, state) == calculateObjective(neighbourBoard, neighbourState):
            neighbourState[randint(0, 100000) % N] = randint(0, 100000) % N
            generateBoard(neighbourBoard, neighbourState)

        step += 1

state = [3, 1, 2, 0]
board = [[0 for _ in range(N)] for _ in range(N)]
generateBoard(board, state)
hillClimbing(board, state)
```

```
Step 1:
  Current State Permutation (columns 0 to 3): [3, 1, 2, 0]
  Attacking queen pairs: 2
  Board layout:
```

```
0 0 0 1
0 1 0 0
0 0 1 0
1 0 0 0
----------------------------------------
Step 2:
  Current State Permutation (columns 0 to 3): [2, 1, 2, 2]
  Attacking queen pairs: 4
  Board layout:
0 0 0 0
0 1 0 0
1 0 1 1
0 0 0 0
----------------------------------------
Step 3:
  Current State Permutation (columns 0 to 3): [3, 1, 2, 2]
  Attacking queen pairs: 2
  Board layout:
0 0 0 0
0 1 0 0
0 0 1 1
1 0 0 0
----------------------------------------
Step 4:
  Current State Permutation (columns 0 to 3): [3, 1, 0, 2]
  Attacking queen pairs: 1
  Board layout:
0 0 1 0
0 1 0 0
0 0 0 1
1 0 0 0
----------------------------------------
Step 5:
  Current State Permutation (columns 0 to 3): [0, 3, 0, 2]
  Attacking queen pairs: 1
  Board layout:
1 0 1 0
0 0 0 0
0 0 0 1
0 1 0 0
----------------------------------------
Step 6:
  Current State Permutation (columns 0 to 3): [1, 3, 0, 2]
  Attacking queen pairs: 0
  Board layout:
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
----------------------------------------
Final solution reached!
```